11 · Scorable negotiation & rational oracles

Siddharth M. Bhatia · siddharth@sidmb.com
July 23, 2026

Contents

The game: issues, score sheets, `GameSpec` 1
Swapping the game: presets 2
Exact solutions & descriptors 4
Rational oracles 5
Playing it: `ScorableNegotiation` 6
Rational agents: `PolicyParticipant` + the strategy zoo 7
Measuring divergence 7
Transcripts & Inspect 8

`interlens.arena.negotiation` is a **multi-party, multi-issue scorable negotiation** stack built for measuring how far an agent (LLM or rule-based) is from *game-theoretically optimal* play — not just whether it closed a deal. It ships a solver-verified game generator, exact normative benchmarks (Pareto frontier, Nash / Kalai–Smorodinsky / egalitarian / utilitarian / max-Nash-welfare points), a swappable **rational-oracle** layer that scores every move, the `ScorableNegotiation` scenario (structured binding offers separated from cheap talk), and a pool of pure-Python rational agents. Every generated instance carries its exact analysis, so per-turn regret and outcome-distance metrics are *exact*, not sampled.

This page is the negotiation-specific companion to 10 · Arena: the game lives in `arena/negotiation/`, the oracle types in `arena/oracles.py`, the scenario in `arena/scenarios/scorable.py`, and the rational agents in `PolicyParticipant`.

The game: issues, score sheets, `GameSpec`

A deal picks one option per issue and is a `Deal = tuple[int, ...]` of option indices. Each party holds a private additive `ScoreSheet` (a value per option,

plus a threshold `τ` — its BATNA); the analysis object is always the **surplus** `x_i(d) = u_i(d) - τ_i`.

```
from interlens.arena.negotiation.space import Issue, DealSpace
from interlens.arena.negotiation.sheets import ScoreSheet, GameSpec

space = DealSpace((Issue("Site", ("North", "South")),
                   Issue("Fund", ("None", "1M", "5M"))))
alice = ScoreSheet("Alice", ((10, 0), (0, 3, 6)), threshold=5.0)
bob   = ScoreSheet("Bob",   ((0, 10), (6, 3, 0)), threshold=5.0)

alice.utility((1, 2))          # 0 + 6  = 6.0   (South, 5M)
alice.surplus((1, 2))          # 6 - 5  = 1.0

spec = GameSpec(space=space, sheets=(alice, bob),
                rounds=4, info="full", chat=True,        # protocol arms
     (DESIGN §3)
                proposer=0, veto=1, min_accept=None,     # agreement rule
     (None = unanimity)
                discount=1.0, breakdown_risk=0.0)        # impatience
     knobs the oracles read
spec.to_json()                 # round-trips via GameSpec.from_json —
     drops into Instance.payload
```

`GameSpec.surplus_matrix()` is the dense `|D| × n` surplus array every solution concept consumes; `feasible_mask()` is the boolean mask of deals that pass the agreement rule (proposer + veto clear, and ≥ `min_accept` parties clear their threshold).

Swapping the game: presets

The situation is a **swappable axis**, like the oracle stack — “two agents play the ultimatum game” is one call. `arena/negotiation/games.py` is a registry mapping a name to a literature-grounded **preset**: a factory returning `(GameSpec, analysis, protocol_cfg)`, the game plus its exact analysis plus the scenario knobs that turn the one `ScorableNegotiation` engine into that game. Presets are *parameterizations, not new engines* — the same deal space, oracle stack, annotation, and atlas run unchanged on any of them.

```
from interlens.arena.negotiation import games

games.PRESETS                                          # {'scorable',
     'ultimatum', 'divide_dollar', 'bilateral_multiissue'}
game, analysis, protocol_cfg = games.make_preset("ultimatum", pie=10,
     n_options=11)
```

```
game.n_parties, analysis["deal_space_size"]            # 2, 11  (one split
    per option)
protocol_cfg                                           # {'single_shot':
    True, 'fixed_proposer': True}

# the arena bridge: a solver-verified Instance + the cfg to play it under
    (reuses generate.build_instance)
instance, protocol_cfg = games.build_preset_instance("divide_dollar",
    n_parties=3, rule="majority")
```

Preset	Game	Rational anchor
`scorable` (default)	the multi-party, multi-issue repaired game (§2 generator; a thin alias, defaults unchanged)	the §4 oracle stack
`ultimatum`	take-it-or-leave-it N=2 pie split (J=1, τ=0, `rounds=1`, single-shot responder-only vote)	SPE: proposer keeps the pie, responder accepts any positive share (Güth et al. 1982 human-rejection contrast; Rubinstein 1982 §5)
`divide_dollar`	N-party discrete-shares split, rotating proposer, multi-round, unanimity/majority	Baron–Ferejohn 1989 / Okada 1996 (`v_i = 1/n` — the equilibrium oracle's own anchor)
`bilateral_multiissue`	DoND-style N=2, J-issue, private values (the generator at `n_parties=2`)	Lewis et al. 2017 lineage

`protocol_cfg` is passed as the scenario `cfg` (or merged into it) so the shared state machine runs the preset's protocol variant. `single_shot` skips

the round-robin and goes straight to the propose→vote forced final; `fixed_proposer` pins the opener seat (no rotation) — both default off, so the standard multi-round game is unchanged. Majority vs unanimity is just `GameSpec.min_accept`. The experiment runner exposes all four as `run.py --game {scorable,ultimatum,divide_dollar,bilateral_multiissue}` (+ `--game-arg KEY=VALUE` preset knobs).

Exact solutions & descriptors

`solutions.analyze(space, sheets)` returns the full, JSON-serializable analysis every generated instance ships with — the normative benchmarks and the score-sheet descriptors that tell you whether the game is actually contestable.

```
from interlens.arena.negotiation import solutions
an = solutions.analyze(space, (alice, bob))
an["deal_space_size"]                  # 6
an["ir_count"], an["pareto_count"]     # |IR set|, |Pareto frontier|
an["ir_pareto_fraction"]               # |IR ∩ Pareto| / |IR| — how near-
     zero-sum the acceptable set is
an["dominated_acceptable_fraction"]    # 1 - that: the score-sheet-repair
     target (leave dominated deals in play)
an["sparsity"], an["pairwise_iou"]     # TMLR descriptors: zero-option
     sparsity, pairwise score overlap
an["ideal_surplus"]                    # per-party best feasible surplus
     b_i (the distance normalizer)
an["solutions"]["nash"]                # {"deal", "named", "surpluses",
     "utilities", "ties", "scale_invariant", ...}
```

The five solution concepts are `nash`, `kalai_smorodinsky`, `egalitarian`, `utilitarian`, `max_nash_welfare`; `all_solutions(space, sheets)` returns them as `SolutionPoint`s (each carries `.deal`, `.surpluses`, `.index`, `.ties`, `.scale_invariant`). Use **nash / kalai_smorodinsky** as the scale-invariant anchors — egalitarian and utilitarian are flagged `scale_invariant=False` (meaningful only on normalized surpluses).

Per-move divergence is a distance in scale-invariant normalized-surplus space:

```
U, tau = spec.utility_matrix(), spec.thresholds
```

```
idx = space.index_of((1, 2))
solutions.distance_to_frontier(U, tau, idx)                # 0.0 iff the
     deal is Pareto-efficient
ks = solutions.all_solutions(space, (alice, bob))["kalai_smorodinsky"]
solutions.distance_to_solution(U, tau, idx, ks.index)      # 0.0 iff the
     deal equals the KS point
```

Rational oracles

An `Oracle` scores every action a seat could take at a decision point and names the best one; the seat's **regret** is `value(best) - value(chosen)` in the game's value units — the centipawn-loss analog for negotiation.

```
from interlens.arena import Oracle, OracleVerdict, OracleRecord, Propose,
     Accept

verdict = OracleVerdict(
    action_values={Accept("O1"): 6.0, Propose((1, 2)): 8.0},   # keyed by
     the typed action
    best=Propose((1, 2)),
    beliefs=None,                                                # a
     belief oracle fills this with its posterior
    flags=[])                                                    # named
     hard-violation markers, e.g. "ir_violation"
verdict.best_value()             # 8.0
verdict.divergence(Accept("O1")) # 8.0 - 6.0 = 2.0  (regret of the chosen
     action, ≥ 0)
verdict.extra                    # free-form per-verdict diagnostics:
     best-response surplus_loss, reservation, v*, …
```

The concrete negotiation oracles live in `arena/negotiation/` (`beliefs.py` exact Bayes over an enumerated type grid, `acceptance.py` optimal-stopping accept/reject, `bestresponse.py` exact expectimax, `equilibrium.py` a Banks–Duggan stationary equilibrium) and subclass this generic ABC. An **inline annotation** scores a seat's actual move against the oracle's best on the same state and is persisted as an `OracleRecord`:

```
rec = OracleRecord.annotation(verdict, round=2, seat="Alice",
     oracle="bestresponse",
                              chosen_action=Accept("O1"), turn_idx=5)
rec.divergence               # 2.0  (best_value - chosen_value) — the per-
     turn regret
```

```
rec.to_json()                # lands in Episode.round_checkpoints
     (identified by a "verdict" key)
```

Playing it: `ScorableNegotiation`

`ScorableNegotiation` (arena/scenarios/scorable.py) plays the game through any participant with **harness-enforced channel separation**: each turn is a private scratchpad, an optional public `message` (cheap talk), and one binding formal move — `Propose` a complete deal (the registry stamps it `P1`, `P2`, …), `Accept` / `Reject` a live offer by id, or `Walk`. Offers are addressed by id so "I accept" is never ambiguous with two proposals on the table.

```
import asyncio
from interlens import APIParticipant, UsageMeter
from interlens.arena import EpisodePool, EpisodeStore
from interlens.arena.scenarios import ScorableNegotiation

scenario = ScorableNegotiation()
instance = scenario.generate_instance(level=0, seed=1)   # payload carries
     the GameSpec, solution = the analyze() dict
meter = UsageMeter(budget=25.0)
player = APIParticipant(name="player", model_id="claude-sonnet-5",
     meter=meter, turn_token_floor=2048)

pool = EpisodePool(EpisodeStore("episodes/"), meter=meter)
episode = asyncio.run(pool.run_episode(scenario, instance,
     arm="moves_chat", participant=player))
episode.outcome        # {"success", "deal": bool, "deal_named": {...},
     "usw"/"esw"/"nsw"/"gini",
                       #  "per_party_surplus", "ir_violations",
     "ceiling_surplus", "offers", ...}
```

Arms: `moves_chat` (binding moves + cheap talk), `moves_only` (the ±communication treatment), `team`, and `solo` (the communication-free control that must not match the multi-agent rate). Situational knobs go through `cfg` (rounds, personas, history window, `show_own_scores`) exactly as in 10 · Arena. Turns persist as `TurnRecord`s with `parsed_action` (`atype` / `deal_named` / `offer`); per-turn oracle annotations persist as `OracleRecord.to_json()` dicts in `Episode.round_checkpoints`.

Rational agents: `PolicyParticipant` + the strategy zoo

A `PolicyParticipant` mounts a pure-Python `policy(state) → Action` as a first-class participant, so a rational agent plays the *same* protocol as an LLM (and can sit across the table from one). The strategy zoo in `arena/negotiation/strategies.py` provides the executable opponents: time-dependent Faratin curves (`TimeDependentPolicy.boulware(...)` concedes near the deadline, `.conceder(...)` concedes early), `MiCROPolicy`, `NaiveTitForTatPolicy`, `ToughPolicy`, and a `BayesianRationalPolicy`, plus AC-next/AC-combi acceptance conditions.

```
from interlens.arena.negotiation.policy_participant
     import PolicyParticipant
from interlens.arena.negotiation.strategies import TimeDependentPolicy

alice_agent = PolicyParticipant(
    name="Alice", policy=TimeDependentPolicy.boulware(), seat=0,
    sheet=alice, space=space, deadline=spec.rounds,
     discount=spec.discount)
# hand alice_agent (and a policy or LLM for bob) to
     EpisodePool.run_episode as the seat participants
```

A rational-vs-rational rollout should land near the frontier in `info="full"` and show calibrated inefficiency in `info="private"` (Myerson–Satterthwaite) — the sanity check before putting LLMs in the seats.

Measuring divergence

The `experiments/rational_agents/` measurement layer reads stored `Episode` + `Instance` and computes the **divergence atlas**: outcome quality (Pareto / NBS / KS distance via the `solutions` functions above, welfare, Gini), per-turn oracle regret (from the `round_checkpoints` `OracleRecord` rows) with the Park et al. no-regret trend tests, concession-curve τ/CRI, and a 12-row failure taxonomy tiered from fully-mechanical (IR violation) to LLM-judge (tactic-style). `annotate.py` writes the per-turn annotations; `analysis/report.py` aggregates a run into per-model per-arm tables + plots. See that experiment's README for the pipeline.

Transcripts & Inspect

Every turn's exact rendered prompt is persisted: the engine records each `SeatRequest.view` into `TurnRecord.view` by default (`EpisodePool(record_views=False)` to disable), so a transcript is faithful even if prompt code later drifts. `arena/export.py` turns a stored run into human-readable transcripts — `export.export_run(episodes_dir, instances_dir, out_dir)` (or `python -m interlens.arena.export --episodes … --instances … --out …`) writes a markdown + self-contained HTML page per episode (game setup, per-turn scratchpad/message/action/oracle-regret, outcome + welfare) plus a run index; it renders old episodes (no `view`) too.

And the scenario is registered as an Inspect task via the preset registry, so `inspect eval` / `inspect view` work out of the box:

```
inspect eval interlens.arena.inspect/scorable --model anthropic/claude-
    sonnet-5 -T game=ultimatum -T arm=moves_only
```

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.